

Advanced Topics in Computer Graphics I

Global Illumination Problem



Prof. Dr. Reinhard Klein

Institute of Computer Science II - Computer Graphics
University of Bonn

April 16, 2025

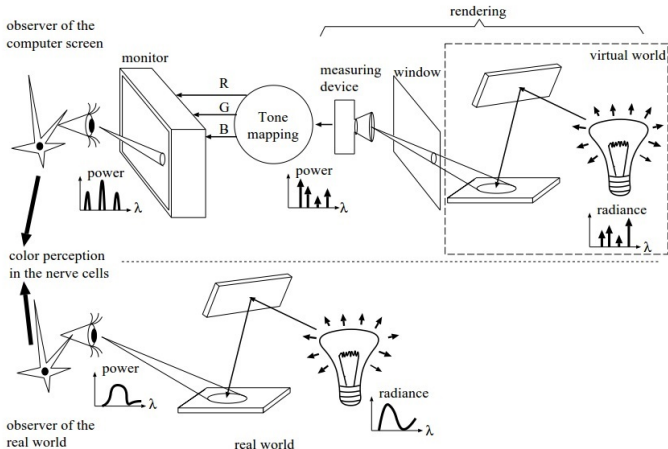




Rendering Task



Task: Try to simulate the real world!

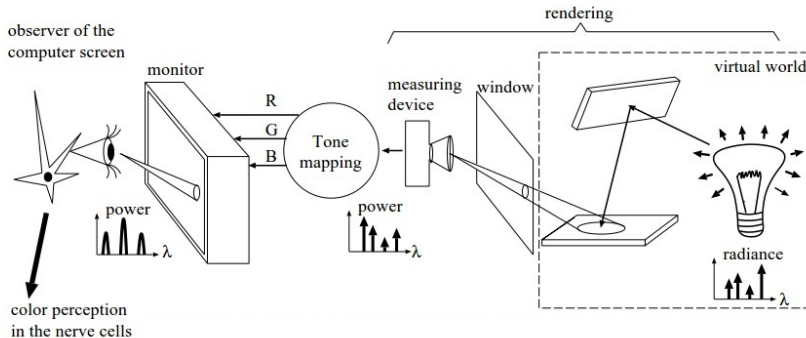




Rendering Task



We already know what light is and how radiance values can be displayed by Low Dynamic Range displays.



Now: Model the light transport in the virtual world ($\rightarrow$ rendering)



Reflection Models and The Rendering Equation





Classical approach: Modeling on different scales (levels of abstraction)

Macroscopic scale

(Geometry, e.g. triangle meshes)



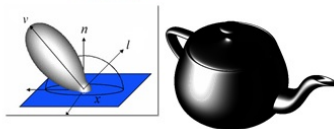
Mesoscopic scale

(e.g. textures, bump maps, displacement maps)



Microscopic scale

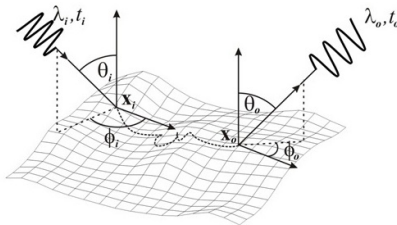
(local reflectance properties, e.g. BRDF)



We will focus on the microscopic scale!



In general, most materials scatter light at their surface in the following way:



θ_i, θ_r - Inclinations

ϕ_i, ϕ_r - Azimuths

$\mathbf{x}_i, \mathbf{x}_r$ - Surface points

λ_i, λ_r - Wave lengths

t_i, t_r - Time of interaction

This scattering is called **Bidirectional Scattering Surface Reflectance Distribution Function (BSSRDF)**.

Note: If a material is refractive/transparent, light is also transmitted inside the surface, so the angles $\theta_{i,o}, \phi_{i,o}$ are defined over the whole sphere $\mathcal{H} = 4\pi$.



Since we want to describe the relation between incoming and scattered light, we need the Volume Rendering Equation for a full solution.

However: If the scences are mainly opaque or transparent surfaces can assume that between the surfaces light travels in vacuum. Furthermore, in this case we can restrict the full solution (BSSRDF is 12-dimensional) in a specific way, to get a simple but still good approximation:

- ▶ $t = \text{const}$: no phosphorescence
 - ▶ $\lambda_i = \lambda_o$: no fluorescence
 - ▶ Finite representation of λ : e.g. $\text{RGB} \rightarrow \mathbb{R}^3$
 - ▶ $\mathbf{x}_i = \mathbf{x}_o$: no subsurface scattering
- ▶ (10-dimensional)
 - ▶ (9-dimensional)
 - ▶ (8-dimensional)
 - ▶ (6-dimensional)

Definition (BSDF¹)

The proportionality constant between incoming irradiance and outgoing radiance is a 6-dimensional function and called *Bidirectional Scattering Distribution Function*

$$f_{BSDF}(\theta_i, \phi_i, \mathbf{x}, \theta_o, \phi_o) = \frac{d\mathbf{L}_o(\theta_i, \phi_i, \mathbf{x}, \theta_o, \phi_o)}{d\mathbf{E}_i(\theta_i, \phi_i, \mathbf{x})} \left[\frac{1}{\text{sr}} \right]$$

¹F. E. Nicodemus et al. *Geometrical considerations and nomenclature for reflectance*. Vol. 160. US Department of Commerce, National Bureau of Standards Washington, USA, 1977.



Example: BRDF vs. BSSRDF



Left : BRDF gives too much contrast and hard shadow boundaries ,
Right : BSSRDF has smoother and more realistic overall appearance



Example: BRDF vs. BSSRDF



Left : BRDF gives too much contrast and hard shadow boundaries ,
Right : BSSRDF has smoother and more realistic overall appearance



Usually, scattered light is subdivided into reflected and transmitted components which are treated separately.

► **Bidirectional Reflectance Distribution Function (BRDF)**

$$f_{BRDF}: \mathcal{H}_i \times \mathcal{S} \times \mathcal{H}_r \rightarrow \mathbb{R}^n$$

where $\mathcal{H}_i$ is the hemisphere from which the incident light comes and $\mathcal{H}_r$ is the hemisphere on which the light is reflected.

► **Bidirectional Transmittance Distribution Function (BTDF)**

$$f_{BTDF}: \mathcal{H}_i \times \mathcal{S} \times \mathcal{H}_t \rightarrow \mathbb{R}^n$$

where $\mathcal{H}_t$ is the hemisphere where light gets transmitted

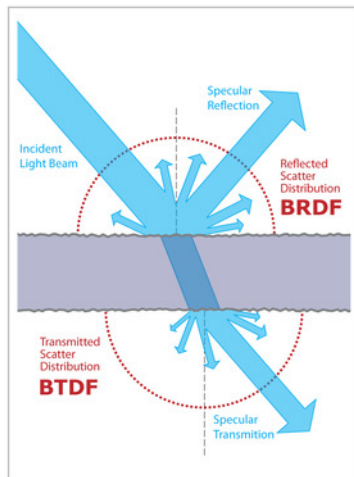
Thus, the BSDF is the union of two BRDF's and two BTDF's (sphere separated into upper and lower hemisphere : $4\pi = \mathcal{H}_{i\ full} = \mathcal{H}_{i\ upper} \cup \mathcal{H}_{i\ lower}$).

Note: This is a very convenient representation of reflection and refraction (1 function instead of 4). Purely reflective or transmissive surfaces are special cases of this formulation.



Question: What is the BSDF (or BRDF/BTDF) of a given material?

- ▶ Optically perfect or smooth surfaces reflect/transmit light according to the physical laws of geometric optics
- ▶ Irregularities induce deviations (light gets scattered also to off-specular directions)
- ▶ Irregularities are often not known in detail → statistical modeling and/or measurements

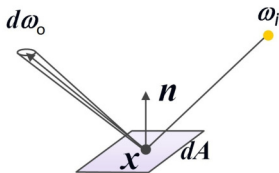




The Rendering Equation



Now, we want to model reflection/refraction by statistical distributions:



For incident flux $d\Phi_i$ (photon) from direction $\omega_i = (\theta_i, \phi_i)$ approaching differential surface element dA at x , we consider the probability of being reflected to differential solid angle $d\omega_o$ in direction $\omega_o = (\theta_o, \phi_o)$.

The corresponding probability density is denoted transfer probability density w :

$$w(\omega_i, x, \omega_o) d\omega_o = \Pr(\text{photon gets scattered to } d\omega_o \mid \text{photon is coming from } \omega_i)$$

Note: We consider all directions from the whole sphere $\mathcal{H} = \mathcal{H}_i = \mathcal{H}_o = 4\pi$. The transfer probability density w has the unit $\left[\frac{1}{\text{sr}}\right]$.



The Rendering Equation



Using this probability density, the expected flux leaving the surface at dA is

$$d^2\Phi_i(\mathbf{x}, \boldsymbol{\omega}_i) \cdot \mathbf{w}(\boldsymbol{\omega}_i, \mathbf{x}, \boldsymbol{\omega}_o) d\omega_o$$

To get the total reflected flux, we need to integrate over all incoming directions $\boldsymbol{\omega}_i$ of the hemisphere $\mathcal{H}_i$

$$\int_{\mathcal{H}_i} d^2\Phi_i(\mathbf{x}, \boldsymbol{\omega}_i) \cdot \mathbf{w}(\boldsymbol{\omega}_i, \mathbf{x}, \boldsymbol{\omega}_o) d\omega_o$$

If the object additionally emits light itself, we have to add an emission term

$$d^2\Phi_o(\mathbf{x}, \boldsymbol{\omega}_o) = d^2\Phi_e(\mathbf{x}, \boldsymbol{\omega}_o) + \int_{\mathcal{H}_i} d^2\Phi_i(\mathbf{x}, \boldsymbol{\omega}_i) \cdot \mathbf{w}(\boldsymbol{\omega}_i, \mathbf{x}, \boldsymbol{\omega}_o) d\omega_o$$

Now, we can use the definition of the radiances

$$d^2\Phi_o(\mathbf{x}, \boldsymbol{\omega}_o) = L_o(\mathbf{x}, \boldsymbol{\omega}_o) \cos(\theta_o) d\omega_o dA$$

$$d^2\Phi_e(\mathbf{x}, \boldsymbol{\omega}_o) = L_e(\mathbf{x}, \boldsymbol{\omega}_o) \cos(\theta_o) d\omega_o dA$$

$$d^2\Phi_i(\mathbf{x}, \boldsymbol{\omega}_i) = L_i(\mathbf{x}, \boldsymbol{\omega}_i) \cos(\theta_i) d\omega_i dA$$



Plugging these equations in and dividing both sides with $dA \cdot d\omega_o \cdot \cos(\theta_o)$, we obtain

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\mathcal{H}_i} \frac{w(\omega_i, \mathbf{x}, \omega_o)}{\cos(\theta_o)} L_i(\mathbf{x}, \omega_i) \cos(\theta_i) d\omega_i$$

If we now set

$$f_{BSDF}(\omega_i, \mathbf{x}, \omega_o) = \frac{w(\omega_i, \mathbf{x}, \omega_o)}{\cos(\theta_o)} \left[\frac{1}{\text{sr}} \right]$$

we get another interpretation of the BSDF. This allows us to model BSDF's (and BRDF's/BTDF's) as statistical distributions!

Definition (Rendering Equation²)

The Rendering Equation is defined as

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\mathcal{H}_i} f_{BSDF}(\omega_i, \mathbf{x}, \omega_o) L_i(\mathbf{x}, \omega_i) \cos(\theta_i) d\omega_i$$

²J. T. Kajiya. "The Rendering Equation". In: *Proceedings of the 13th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '86. ACM, 1986, pp. 143–150.



Intuitively, the Rendering Equation

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\mathcal{H}_i} f_{BSDF}(\omega_i, \mathbf{x}, \omega_o) L_i(\mathbf{x}, \omega_i) \cos(\theta_i) d\omega_i$$

states that in case of energy balance at a surface, the reflected radiance L_o must be equal to the emitted radiance L_e plus the fraction of incoming radiance L_i from arbitrary directions ω_i that hits the point $\mathbf{x}$ and is reflected/refracted towards the direction ω_o .

The second term is also called the **Reflection Equation** when restricting to BRDF's:

$$L_o(\mathbf{x}, \omega_o) = \int_{\mathcal{H}_i} f_{BRDF}(\omega_i, \mathbf{x}, \omega_o) L_i(\mathbf{x}, \omega_i) \cos(\theta_i) d\omega_i$$

If we use $L_o(\mathbf{x}, \omega_o) = \int_{\mathcal{H}_i} dL_o(\omega_i, \mathbf{x}, \omega_o)$ and $dE_i(\mathbf{x}, \omega_i) = L_i(\mathbf{x}, \omega_i) \cos(\theta_i) d\omega_i$, we again get the following interpretation of the BSDF

$$f_{BSDF}(\omega_i, \mathbf{x}, \omega_o) = \frac{dL_o(\omega_i, \mathbf{x}, \omega_o)}{dE_i(\mathbf{x}, \omega_i)} \left[\frac{1}{\text{sr}} \right]$$

So we have a physical and a statistical interpretation of BSDFs.



The Light Transport Equation



If we take a closer look at the rendering equation

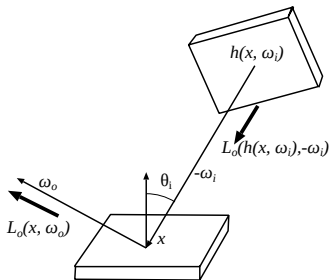
$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_{\mathcal{H}_i} f_{BSDF}(\omega_i, x, \omega_o) L_i(x, \omega_i) \cos(\theta_i) d\omega_i$$

we realize that in case of vacuum where no participating media is present the incoming light $L_i(x, \omega_i)$ can be written as outgoing light.

Incident light is nothing else than outgoing light when tracing back the ray in direction ω_i until it hits the scene again:

$$L_i(x, \omega_i) = L_o(h(x, \omega_i), -\omega_i)$$

The **ray-casting function** $h(x, \omega_i)$ calculates the scenepoint hit by the ray from x in direction ω_i . So we get a recursive coupling in the rendering equation. In this form the rendering equation is also called **Light Transport Equation**.



$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_{\mathcal{H}_i} f_{BSDF}(\omega_i, x, \omega_o) L_o(h(x, \omega_i), -\omega_i) \cos(\theta_i) d\omega_i$$



The Light Transport Equation



Sometimes it will be easier to use a surface form of the **Light Transport Equation (LTE)** that integrates over surface points instead of directions for incident radiance³.

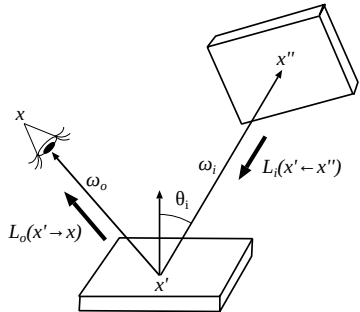
Let's start by first defining the exitant radiance L_o from a surface point x' to another surface point x (thus in direction $\omega_o = x - x'$) as

$$L_o(x' \rightarrow x) := L_o(x', x - x') = L_o(x', \omega_o).$$

Analogously the incident radiance L_i of a surface point x' from x'' is

$$L_i(x' \leftarrow x'') := L_i(x', x' - x'') = L_i(x', \omega_i).$$

Note, this only holds for pairs of points that are mutually visible.



Accordingly, with $\omega_i = x'' - x'$ and $\omega_o = x - x'$, we can define our BSDF as

$$f_{BSDF}(x'' \rightarrow x' \rightarrow x) := f_{BSDF}(\omega_i, x', \omega_o).$$

³M. Pharr et al. *Physically based rendering: From theory to implementation*. Morgan Kaufmann, 2016.

URL: <http://www.pharr-book.org/>



The Light Transport Equation



Furthermore, we have to change the integral over directions into an integral over area. This is done by using $d\omega_i = \frac{\cos(\theta_i)}{\|x'' - x'\|^2} dA(x'')$ as seen in previous chapter. In order to account for occluded points we also add a visibility term $V(x'' \leftrightarrow x')$ which is 1 if the points are mutually visible and 0 otherwise.

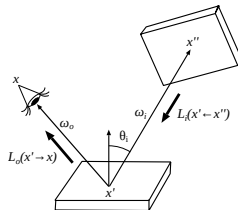
Plugging everything together and writing L instead of L_o for brevity we arrive at

$$L(x' \rightarrow x) = L_e(x' \rightarrow x) + \int_S f_{BSDF}(x'' \rightarrow x' \rightarrow x) L(x' \leftarrow x'') G(x'' \leftrightarrow x') dA(x'')$$

where S contains the scene's surfaces and the geometry term $G(x'' \leftrightarrow x')$ is defined as

$$G(x'' \leftrightarrow x') := h(x'' \leftrightarrow x') \frac{\cos(\theta_i) \cos(\theta_o)}{\|x'' - x'\|^2}$$

to improve readability. Note, the the visibility term $V(x'' \leftrightarrow x')$ can be evaluated using the ray-casting function $h(x', x'' - x')$.





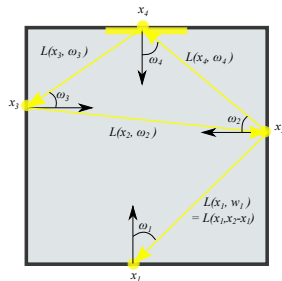
The The Light Transport Equation as integral equation



Independent of the integration over directions or surface points the **LTE** is an integral equation, where the unknown function $L(x, \omega_o)$ on the left-hand side is contained in the integral on the right hand side.

$$L(x, \omega_o) = L_e(x, \omega_o) + \int_{\mathcal{H}_i} f_{BSDF}(\omega_i, x, \omega_o) L(h(x, \omega_i), -\omega_i) \cos(\theta_i) d\omega_i$$

As shown in the simple 2D example the solution of the rendering equation, i.e. the knowledge of the function $L(x, \omega_o)$ allows to query at each surface point x its brightness as seen from direction $-\omega_o$.



Rendering challenge: solve this integral equation efficiently⁴⁵!

⁴L. Szirmay-Kalos. *Monte Carlo Methods in Global Illumination - Script WS 1999/2000*. Vienna University of Technology, 2000.

⁵L. Szirmay-Kalos. *Monte Carlo Methods in Global Illumination - Photo-realistic Rendering with Randomization*. Saarbrücken, Germany, Germany: VDM Verlag, 2008.



In the case of general reflection properties of the scene surfaces the solution of the LTE requires more efforts.

$$L(x, \omega_o) = L_e(x, \omega_o) + \int_{\mathcal{H}_i} f_{BSDF}(\omega_i, x, \omega_o) L(h(x, \omega_i), -\omega_i) \cos(\theta_i) d\omega_i$$

To write it more concise we define an integral operator

$$(TL)(x, \omega_o) := \int_{\mathcal{H}_i} f_{BSDF}(\omega_i, x, \omega_o) L(h(x, \omega_i), -\omega_i) \cos(\theta_i) d\omega_i$$

Thus, we can express the rendering equation in the following way:

$$L = L_e + TL$$

If we rearrange the terms, we get the following relation

$$L = L_e + TL$$

$$L - TL = L_e$$

$$(I - T)L = L_e$$

$$L = (I - T)^{-1} L_e$$

So we need to invert $(I - T)$ to get a full solution.



Radiosity Method





A matrix representation for $I - T$:

If all the surfaces in the scene reflect purely diffuse (Lambertian)

$$f_{BSDF}(\omega_i, \mathbf{x}, \omega_o) = f_{BSDF}(\mathbf{x}),$$

the reflected radiance $\mathbf{L}(\mathbf{x}, \omega_o) = \mathbf{L}_o(\mathbf{x}, \omega_o)$ as well as the emitted radiance $\mathbf{L}_e(\mathbf{x}, \omega_o)$ is independent of the outgoing light direction:

$$\mathbf{L}_{\{e,o\}}(\mathbf{x}, \omega_o) = \mathbf{L}_{\{e,o\}}(\mathbf{x}).$$

In this case we consider the **radiosity (Radiometric Quantity)** instead of radiance: While irradiance $\mathbf{E}$ is radiant power per unit area *entering* a surface, radiosity $\mathbf{M}$ is total radiant power per unit area *leaving* a surface. For a purely diffuse surface, we get

$$\begin{aligned} \mathbf{M}(\mathbf{x}) &= \frac{d\Phi}{dA} = \int_{\phi=0}^{2\pi} \int_{\theta=0}^{\frac{\pi}{2}} \mathbf{L}(\mathbf{x}) \cos(\theta) \sin(\theta) d\theta d\phi \\ &= \mathbf{L}(\mathbf{x}) \int_{\phi=0}^{2\pi} \int_{\theta=0}^{\frac{\pi}{2}} \cos(\theta) \sin(\theta) d\theta d\phi = \mathbf{L}(\mathbf{x}) \pi. \end{aligned}$$



Starting with the the surface notation of the LTE

$$L(\mathbf{x}' \rightarrow \mathbf{x}) = L_e(\mathbf{x}' \rightarrow \mathbf{x}) + \int_S f_{BSDF}(\mathbf{x}'' \rightarrow \mathbf{x}' \rightarrow \mathbf{x}) L(\mathbf{x}' \leftarrow \mathbf{x}'') G(\mathbf{x}'' \leftrightarrow \mathbf{x}') dA(\mathbf{x}'')$$

and assuming that all surfaces are diffuse the radiance and the BSDF depends on the surface location $\mathbf{x}$ only

$$L(\mathbf{x}) = L_e(\mathbf{x}) + f_{BSDF}(\mathbf{x}) \int_S L(\mathbf{x}') G(\mathbf{x} \leftrightarrow \mathbf{x}') dA(\mathbf{x}'),$$

an integrating over all possible outgoing light directions (full hemisphere) yields

$$\underbrace{L(\mathbf{x}) \pi}_{B(\mathbf{x})} = \underbrace{L_e(\mathbf{x}) \pi}_{E(\mathbf{x})} + \underbrace{f_{BSDF}(\mathbf{x}) \pi}_{r(\mathbf{x})} \int_S \underbrace{L(\mathbf{x}')}_{B(\mathbf{x}')/\pi} G(\mathbf{x} \leftrightarrow \mathbf{x}') dA(\mathbf{x}').$$



Definition (Radiosity Equation^{6,7})

The Radiosity Equation is defined as

$$B(x) = E(x) + r(x) \int_S B(x') \frac{G(x \leftrightarrow x')}{\pi} dA(x')$$

Note, that in the Radiosity Equation the integral operator acts on the radiosity instead of the radiance.

⁶C. M. Goral et al. "Modeling the interaction of light between diffuse surfaces". In: *ACM SIGGRAPH computer graphics* 18.3 (1984), pp. 213–222.

⁷M. F. Cohen et al. "A progressive refinement approach to fast radiosity image generation". In: *Proceedings of the 15th annual conference on Computer graphics and interactive techniques*. 1988, pp. 75–84.



Solving the Radiosity Equation



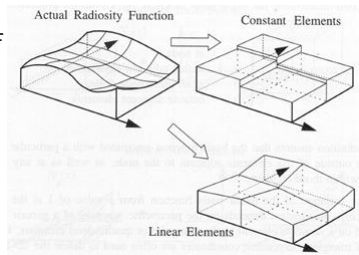
$$B(x) = E(x) + r(x) \int_S B(x') \frac{G(x \leftrightarrow x')}{\pi} dA(x').$$

Approximating radiosity with a linear combination of basis functions N_i leads to a vector representation of the radiosity

$$B(x) \approx \hat{B}(x) := \sum_{i=1}^n B_i N_i(x)$$

Plugging this approximation into the radiosity equation yields

$$\hat{B}(x) = E(x) + r(x) \int_S \hat{B}(x') \frac{G(x \leftrightarrow x')}{\pi} dA(x').$$





Calculate Residual

$$\mathbf{R}(\mathbf{x}) = \hat{\mathbf{B}}(\mathbf{x}) - \mathbf{E}(\mathbf{x}) - \mathbf{r}(\mathbf{x}) \int_S \hat{\mathbf{B}}(\mathbf{x}') \frac{\mathbf{G}(\mathbf{x} \leftrightarrow \mathbf{x}')}{\pi} dA(\mathbf{x}').$$

If $\mathbf{B}(\mathbf{x}) = \hat{\mathbf{B}}(\mathbf{x})$, then $\mathbf{R}(\mathbf{x}) = 0$.

Minimize $\mathbf{R}(\mathbf{x})$ by using the Galerkin method of weighted residuals:

$$\begin{aligned} \langle \mathbf{N}_i(\mathbf{x}), \mathbf{R}(\mathbf{x}) \rangle &= \left\langle \mathbf{N}_i(\mathbf{x}), \hat{\mathbf{B}}(\mathbf{x}) - \mathbf{E}(\mathbf{x}) - \mathbf{r}(\mathbf{x}) \int_S \hat{\mathbf{B}}(\mathbf{x}') \frac{\mathbf{G}(\mathbf{x} \leftrightarrow \mathbf{x}')}{\pi} dA(\mathbf{x}') \right\rangle \\ &= \int_S \mathbf{N}_i(\mathbf{x}) \hat{\mathbf{B}}(\mathbf{x}) dA(\mathbf{x}) - \int_S \mathbf{N}_i(\mathbf{x}) \mathbf{E}(\mathbf{x}) dA(\mathbf{x}) \\ &\quad - \int_S \mathbf{N}_i(\mathbf{x}) \mathbf{r}(\mathbf{x}) \int_S \hat{\mathbf{B}}(\mathbf{x}') \frac{\mathbf{G}(\mathbf{x} \leftrightarrow \mathbf{x}')}{\pi} dA(\mathbf{x}') dA(\mathbf{x}) \\ &= 0 \forall i \Rightarrow \mathbf{R}(\mathbf{x}) = 0 \end{aligned}$$



Solving the Radiosity Equation



Substituting $\hat{B}(x) = \sum_{i=1}^n B_i N_i(x)$ delivers:

$$\begin{aligned} & \int_S N_i(x) \sum_{j=1}^n B_j N_j(x) dA(x) - \int_S N_i(x) E(x) dA(x) \\ & \quad - \int_S N_i(x) r(x) \int_S \sum_{j=1}^n B_j N_j(x') \frac{G(x \leftrightarrow x')}{\pi} dA(x') dA(x) \\ &= \sum_{j=1}^n B_j \int_S N_i(x) N_j(x) dA(x) - \int_S N_i(x) E(x) dA(x) \\ & \quad - \sum_{j=1}^n B_j \int_S N_i(x) r(x) \int_S N_j(x') \frac{G(x \leftrightarrow x')}{\pi} dA(x') dA(x) \\ &= 0 \forall i \end{aligned}$$



Reorganizing gives (for all i)

$$\begin{aligned} \sum_{j=1}^n B_j \underbrace{\left(\int_S N_i(\mathbf{x}) N_j(\mathbf{x}) dA(\mathbf{x}) - \int_S N_i(\mathbf{x}) r(\mathbf{x}) \int_S N_j(\mathbf{x}') \frac{G(\mathbf{x} \leftrightarrow \mathbf{x}')}{\pi} dA(\mathbf{x}') dA(\mathbf{x}) \right)}_{K_{ij}} \\ = \underbrace{\int_S N_i(\mathbf{x}) E(\mathbf{x}) dA(\mathbf{x})}_{E_i} \end{aligned}$$

which can be summed up into the linear equation system

$$\begin{pmatrix} K_{11} & \cdots & K_{1n} \\ \vdots & \ddots & \vdots \\ K_{n1} & \cdots & K_{nn} \end{pmatrix} \begin{pmatrix} B_1 \\ \vdots \\ B_n \end{pmatrix} = \begin{pmatrix} E_1 \\ \vdots \\ E_n \end{pmatrix}$$

The matrix $\mathbf{K}$ is the matrix representation the operator $(\mathbf{I} - \mathbf{T})$. It is determined by the choice of the basis functions in the radiosity function space.



Solving the Radiosity Equation



$$\sum_{j=1}^n B_j \left(\int_S N_i(\mathbf{x}) N_j(\mathbf{x}) dA(\mathbf{x}) - \int_S N_i(\mathbf{x}) \mathbf{r}(\mathbf{x}) \int_S N_j(\mathbf{x}') \frac{G(\mathbf{x} \leftrightarrow \mathbf{x}')}{\pi} dA(\mathbf{x}') dA(\mathbf{x}) \right) = \int_S N_i(\mathbf{x}) \mathbf{E}(\mathbf{x}) dA(\mathbf{x})$$

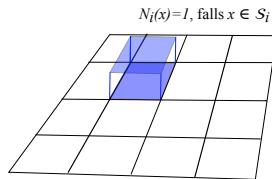
For piecewise constant basis functions we get:

$$\int_S N_i(\mathbf{x}) \mathbf{E}(\mathbf{x}) dA(\mathbf{x}) = \mathbf{E}_i A_i$$

$$\int_S N_i(\mathbf{x}) N_j(\mathbf{x}) dA(\mathbf{x}) = \begin{cases} A_i & i = j \\ 0 & \text{else} \end{cases}$$

$$N_i(\mathbf{x}) := \begin{cases} 1 & \mathbf{x} \text{ is in element } S_i \\ 0 & \text{else} \end{cases}$$

$$\int_S N_i(\mathbf{x}) \mathbf{r}(\mathbf{x}) \int_S N_j(\mathbf{x}') \frac{G(\mathbf{x} \leftrightarrow \mathbf{x}')}{\pi} dA(\mathbf{x}') dA(\mathbf{x}) = \mathbf{r}_i \int_{S_i} \int_{S_j} \frac{G(\mathbf{x} \leftrightarrow \mathbf{x}')}{\pi} dA(\mathbf{x}') dA(\mathbf{x})$$





Solving the Radiosity Equation



Substituting gives for all i :

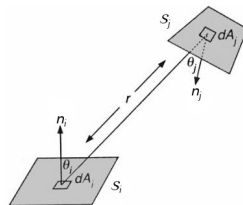
$$\begin{aligned} \sum_{j=1}^n B_j A_i \left(\delta_{ij} - r_i \frac{1}{A_i} \int_{S_i} \int_{S_j} \frac{G(\mathbf{x} \leftrightarrow \mathbf{x}')}{\pi} dA(x') dA(x) \right) &= E_i A_i \\ \Leftrightarrow \sum_{j=1}^n B_j \left(\delta_{ij} - r_i \frac{1}{A_i} \int_{S_i} \int_{S_j} \frac{G(\mathbf{x} \leftrightarrow \mathbf{x}')}{\pi} dA(x') dA(x) \right) &= E_i \\ \Leftrightarrow \sum_{j=1}^n B_j \underbrace{(\delta_{ij} - r_i F_{ij})}_{K_{ij}} &= E_i \end{aligned}$$

with **form factor**

$$F_{ij} := \frac{1}{A_i} \int_{S_i} \int_{S_j} \frac{G(\mathbf{x} \leftrightarrow \mathbf{x}')}{\pi} dA(x') dA(x)$$

and

$$\delta_{ij} := \begin{cases} 1 & i = j \\ 0 & \text{else.} \end{cases}$$





Solving the Radiosity Equation



$$\sum_{j=1}^n B_j \underbrace{(\delta_{ij} - r_i F_{ij})}_{K_{ij}} = E_i$$

By multiplying the brackets and isolating the resulting B_i we finally arrive at the discrete form of the *Radiosity Equation*:

$$B_i = E_i + r_i \sum_{j=1}^n B_j F_{ij} \forall i.$$

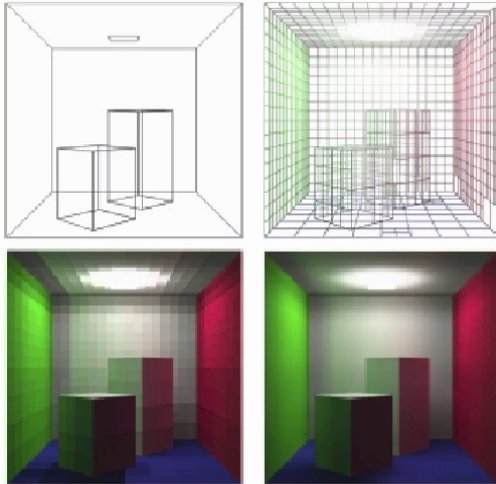
It tells us, that the radiosity on element S_i is the emission on element S_i plus the sum of radiosities on all other elements S_j scaled with r_i and weighted by the form factors F_{ij} .



Solving the Radiosity Equation



Example:





Solving the Radiosity Equation



Example:



Figure: Illumination calculation with the RadioLabsystem, Ralf Sonntag, Tübingen⁸



General case In the case of general reflection properties of the scene surfaces the solution of the LTE requires more efforts.

If we exploit the recursive nature of the LTE again and again, we get

$$L = L_e + TL = L_e + T(L_e + TL) = L_e + TL_e + T^2L$$

Repeating this n -times, the original equation can be expanded to

$$L = \sum_{i=0}^n T^i L_e + T^{n+1} L$$

If we now assume that the integral operator is a contraction, i.e. $\|T\| < 1$, then we get the Neumann series

$$L = \sum_{i=0}^{\infty} T^i L_e$$

which has the solution

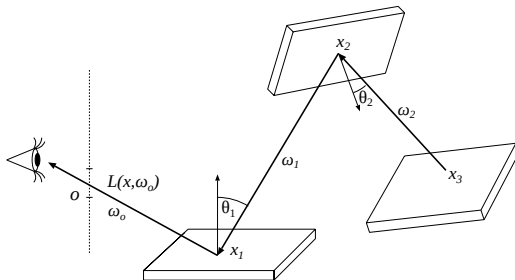
$$L = \sum_{i=0}^{\infty} T^i L_e = (I - T)^{-1} L_e$$

Note: In case of purely diffuse surfaces the result is exactly the same as in the radiosity method.

Interpretation: $T^0 L_e$ gathers light energy of the direct emission, $T^1 L_e$ that after one reflection, $T^2 L_e$ that after two reflections, ...

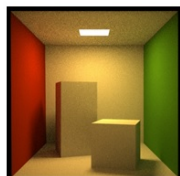
Problem: Even computing this for $n = 2$ is computationally expensive:

$$\begin{aligned} T^2 L_e &= \int_{\mathcal{H}_1} f_{BSDF}(\omega_1, \mathbf{x}_1, \omega_o) L_e(\mathbf{h}(\mathbf{x}, \omega_1), -\omega_1) \cos(\theta_1) d\omega_1 \\ &= \int_{\mathcal{H}_1} f_{BSDF}(\omega_1, \mathbf{x}_1, \omega_o) \cos(\theta_1) \left(\int_{\mathcal{H}_2} f_{BSDF}(\omega_2, \mathbf{x}_2, \omega_1) L_e(\mathbf{h}(\mathbf{x}, \omega_2), -\omega_2) \cos(\theta_2) d\omega_2 \right) d\omega_1 \end{aligned}$$



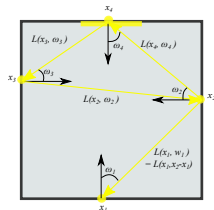
Solving The Rendering Equation

But there is hope: T is assumed to be a contraction. This means the contributions from higher order reflections T^n become smaller and smaller:

 L_e  $T \circ L_e$  $T \circ T \circ L_e$  $T \circ T \circ T \circ L_e$  L_e  $L_e + T \circ L_e$  $L_e + \dots + T^2 \circ L_e$  $L_e + \dots + T^3 \circ L_e$

The infinite sum over the path that are themselves points in a high dimensional **path space** can be written compactly as

$$L(x_1 \rightarrow x_0) = \sum_{n=1}^{\infty} P(\bar{x}_n).$$



$P(\bar{x}_n)$ gives the amount of radiance scattered over a path $\bar{x}_n$ with $n + 1$ vertices

$$\bar{x}_n = x_0, x_1, \dots, x_n,$$

where x_0 is on the film plane or front lens element and x_n is on a light source, and

$$P(\bar{x}_n) = \underbrace{\int_S \int_S \cdots \int_S}_{n-1} L_e(x_n \rightarrow x_{n-1}) \times \left(\prod_{i=1}^{n-1} f_{BSDF}(x_{i+1} \rightarrow x_i \rightarrow x_{i-1}) G(x_{i+1} \leftrightarrow x_i) \right) dA(x_2) \cdots dA(x_n).$$



Let

$$f_{BSDF}(x_{i+1} \rightarrow x_i \rightarrow x_{i-1}) \text{ and } G(x_{i+1} \leftrightarrow x_i) = h(x_{i+1} \leftrightarrow x_i) \frac{\cos(\theta_i) \cos(\theta_o)}{\|x_{i+1} - x_i\|^2}$$

be the BSDF and the geometry term at the i -th vertex x_i of a path starting at the camera.

Definition (Throughput)

The product of a path's BSDF and geometry terms

$$T(\bar{x}_n) = \prod_{i=1}^{n-1} f_{BSDF}(x_{i+1} \rightarrow x_i \rightarrow x_{i-1}) G(x_{i+1} \leftrightarrow x_i)$$

is called **throughput** of the path.

It describes the fraction of radiance from the light source that arrives at the camera after all of the scattering at vertices between them.

We will see during the lecture how this formulation leads to practical Monte Carlo based light transport algorithms.



The Path-Space Measurement Equation



The **measurement equation** describes the value of an abstract measurement that is found by integrating over some set of rays carrying radiance. When computing the value of a pixel in the image, we can write the pixel's value as an integral over points on the film plane of a weighting function times the incident radiance along the corresponding camera rays:

$$\begin{aligned} I_j &= \int_{\mathcal{S}_{film}} \int_{\Omega} W_e^{(j)}(x_{film}, \omega_i) L_i(x_{film}, \omega_i) |\cos(\theta)_i| d\omega_i d\mathcal{S}(x_{film}), \\ &= \int_{\mathcal{S}_{film}} \int_{\Omega} W_e^{(j)}(x_0 \rightarrow x_1) L(x_0 \rightarrow x_1) G(x_0 \leftrightarrow x_1) d\mathcal{S}(x_1) d\mathcal{S}(x_0), \end{aligned}$$

where I_j is the measurement for the j th pixel and x_0 is a point on the film. In this setting, the $W_e^{(j)}(x_0 \rightarrow x_1)$ term is the product of the filter function around the pixel f_j and a delta function that selects the appropriate camera ray direction of the sample from $x_0, \omega_{camera}(x_1)$:

$$W_e^{(j)}(x_0 \rightarrow x_1) = f_j(x_0) \delta(h(x_0, \omega_{camera}(x_1)) - x_1).$$

The value described by the term W_e is known as the **importance** for the ray between x_0 and x_1 in the scene.

If we expand the $P(\bar{x}_n)$ terms of the LTE sum, we have :

$$\begin{aligned}
 I_j &= \int_{\mathcal{S}_{film}} \int_{\mathcal{S}} W_e^{(j)}(x_0 \rightarrow x_1) L(x_0 \rightarrow x_1) G(x_0 \leftrightarrow x_1) d\mathcal{S}(x_1) d\mathcal{S}(x_0) \\
 &= \sum_i \int_{\mathcal{S}_{film}} \int_{\mathcal{S}} W_e^{(j)}(x_0 \rightarrow x_1) P(\bar{x}_n) G(x_0 \leftrightarrow x_1) d\mathcal{S}(x_1) d\mathcal{S}(x_0) \\
 &= \sum_i \underbrace{\int_{\mathcal{S}} \cdots \int_{\mathcal{S}}}_{n+1} W_e^{(j)}(x_0 \rightarrow x_1) T(\bar{x}_i) L_e(x_{i+1} \rightarrow x_i) \\
 &\quad G(x_0 \leftrightarrow x_1) d\mathcal{S}(x_{i+1}) \cdots d\mathcal{S}(x_0),
 \end{aligned}$$

where $T(\bar{x}_i)$ is the path throughput function.



The Path-Space Measurement Equation



Observing the symmetric way in which the emitted radiance L_e (quantifying the light source emission profile) and the weighting function $W_e^{(j)}$ (quantifying the camera's sensitivity profile for a pixel) appear in the equation

$$I_j = \sum_i \underbrace{\int_S \cdots \int_S}_{n+1} W_e^{(j)}(x_0 \rightarrow x_1) T(\bar{x}_i) L_e(x_{i+1} \rightarrow x_i) G(x_0 \leftrightarrow x_1) dS(x_{i+1}) \cdots dS(x_0),$$

we infer that the concepts of emission and measurement are mathematically interchangeable:

- **Interpretation 1:** Light could be emitted from light sources, bounce around the scene, and arrive at a sensor where describes its contribution to the measurement
- **Interpretation 2:** the sensor can be thought of emitting an imaginary quantity (importance) that creates a measurement when it reaches a light source.

By simply swapping the role of cameras and light sources in this way, the **particle tracing** method, which traces rays from the light sources to recursively estimate the incident importance arriving on surfaces, was introduced.

This is not a particularly useful rendering technique on its own, but it constitutes an essential ingredient of further methods such as **bidirectional path tracing** and **photon mapping**.



Classification of Solution Techniques

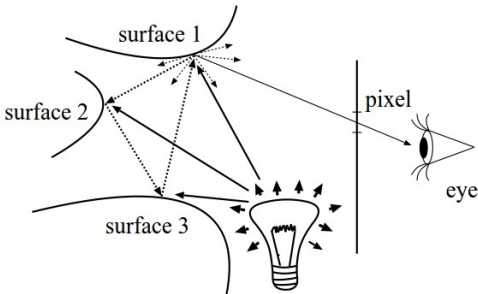




Observation: If we cannot fully solve the Rendering Equation, we need to approximate it

$$L = L_e + TL$$

Idea: Only compute those parts of the integral that contribute most to the solution

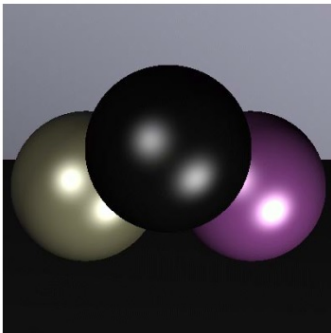




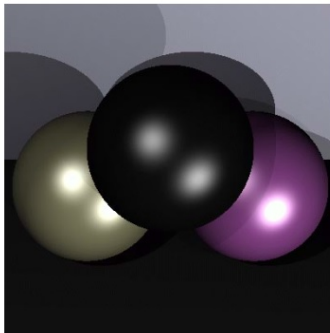
Method 1: Local illumination methods (Ray Casting)

$$L(x, \omega_o) = L_e(x, \omega_o) + \int_{\mathcal{H}_i} f_{BRDF}(\omega_i, x, \omega_o) L_{els}(h(x, \omega_i), -\omega_i) \cos(\theta_i) d\omega_i$$

where L_{els} may be a simplification of the emission function L_e at light sources and only the BRDF is considered.



local illumination method

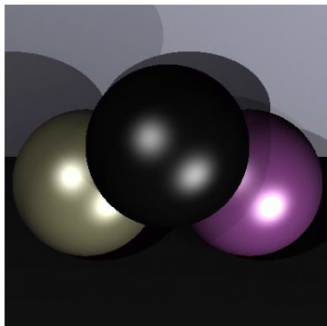


local illumination method with shadow computation



Method 2: Recursive Ray Tracing (Whitted Ray Tracing)

$$L(x, \omega_o) = L_e(x, \omega_o) + \int_{\mathcal{H}_i} f_{BRDF}(\omega_i, x, \omega_o) L_{els}(\mathbf{h}(x, \omega_i), -\omega_i) \cos(\theta_i) d\omega_i$$
$$+ \underbrace{k_{ir}(\omega_{ir}, x, \omega_o) L(\mathbf{h}(x, \omega_{ir}), -\omega_{ir})}_{\text{ideal reflection}} + \underbrace{k_{it}(\omega_{it}, x, \omega_o) L(\mathbf{h}(x, \omega_{it}), -\omega_{it})}_{\text{ideal transmission}}$$



local illumination method with shadow computation



recursive ray-tracing



Method 3a: Radiosity Method

$$L(\mathbf{x}) = L_e(\mathbf{x}) + \int_{\mathcal{H}_i} f_{BSDF}(\mathbf{x}) L(\mathbf{h}(\mathbf{x}, \boldsymbol{\omega}_i)) \cos(\theta_i) d\boldsymbol{\omega}_i$$

Assumption:

All Surfaces in the scene are purely diffuse (lambertian). Thus, reflected luminance is independent of the outgoing light direction.

$$f_{BSDF}(\boldsymbol{\omega}_i, \mathbf{x}, \boldsymbol{\omega}_o) = f_{BSDF}(\mathbf{x})$$





Method 3b: Global solution

$$L(\mathbf{x}, \boldsymbol{\omega}_o) = L_e(\mathbf{x}, \boldsymbol{\omega}_o) + \int_{\mathcal{H}_i} f_{BSDF}(\boldsymbol{\omega}_i, \mathbf{x}, \boldsymbol{\omega}_o) L(\mathbf{h}(\mathbf{x}, \boldsymbol{\omega}_i), -\boldsymbol{\omega}_i) \cos(\theta_i) d\boldsymbol{\omega}_i$$



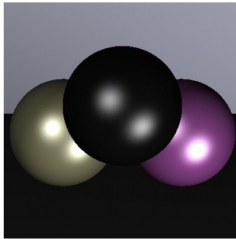
recursive ray-tracing



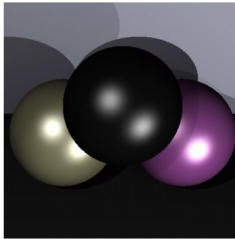
global illumination method



Comparison:



local illumination method



local illumination method with shadow computation



recursive ray-tracing



global illumination method



Idea: Characterize all scattering events (radiance transfer) of a light path starting at the light source (**L**) to the eye (**E**) in one string:⁹

- ▶ E is the eye
- ▶ L is the lightsource
- ▶ D is a diffuse reflection or refraction
- ▶ S is a specular reflection or refraction
- ▶ * is the sign of iteration, that is, it can mean 0, 1, 2, ... applications of the given operator
- ▶ [] represents optionality
- ▶ | means selection

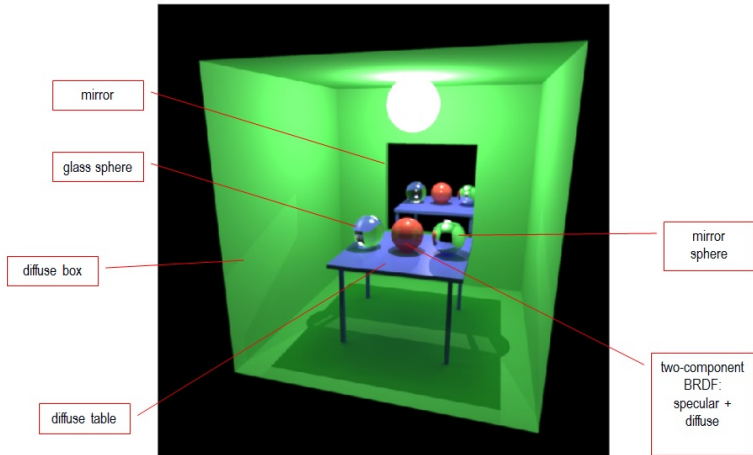
Examples:

- | | |
|---|------------------------------------|
| ▶ Global Illumination: L[D S]*E | ▶ Radiosity Method: L[D]*E |
| ▶ Recursive Ray Tracing: L[DS*]E | ▶ Local Illumination: L[D]E |

⁹P. S. Heckbert. "Adaptive Radiosity Textures for Bidirectional Ray Tracing". In: *Proceedings of the 17th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '90. ACM, 1990, pp. 145–154.



Example:

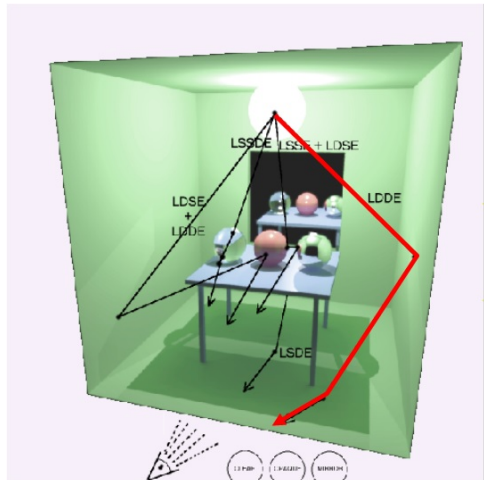




Example:

LDDE

- Light is diffusely reflected by wall and floor



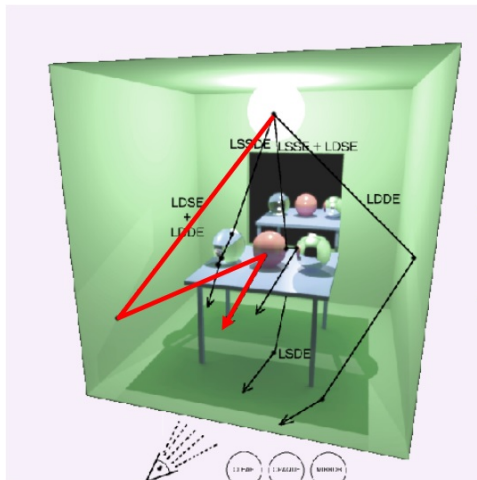


Example:

LDSE+LDDE

- ▶ Diffuse reflection at wall
- ▶ Specular and diffuse reflection at sphere

Note: Eye sees only shaded region of the sphere (no direct light)

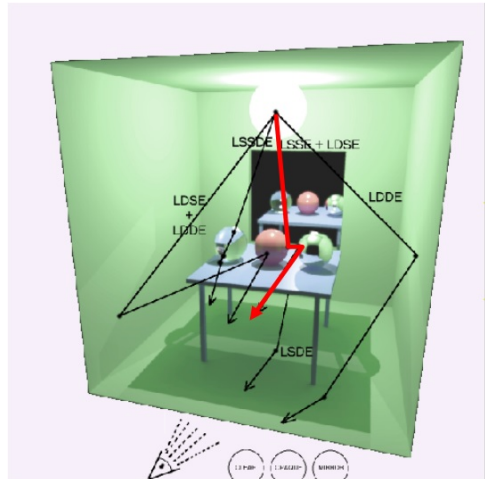




Example:

LSSE+LDSE

- ▶ Eye sees ideally specular reflection from mirror sphere
- ▶ Red sphere reflects specular and diffuse

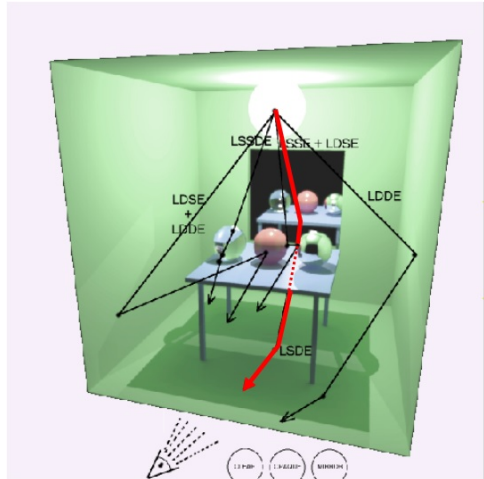




Example:

LSDE

- Shadow region below table is brighter than the rest (caused by light reflected from the mirror)

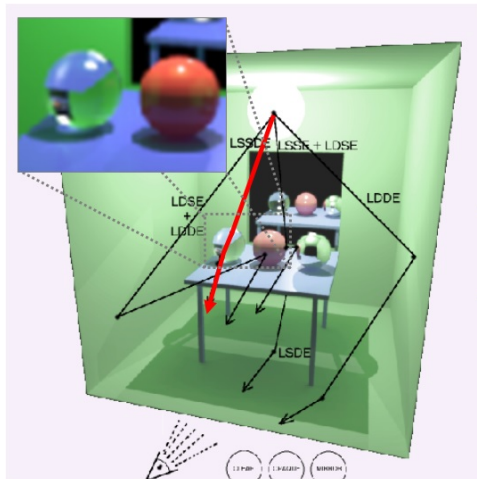




Example:

LSSDE

- ▶ Ray gets refracted while entering glass sphere (S)
- ▶ Ray gets refracted while leaving sphere (S)
- ▶ Ray is diffusely reflected by table (D)

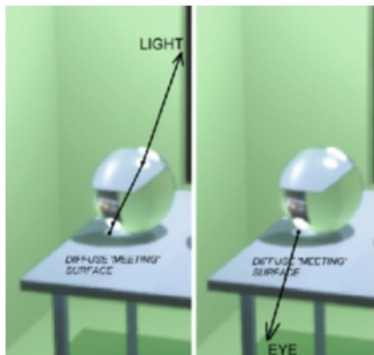




Example:

LSSDE

- ▶ Glass sphere “concentrates” light to a small region on diffuse table: bright spot = local increase of irradiance! (“refraction caustic”)

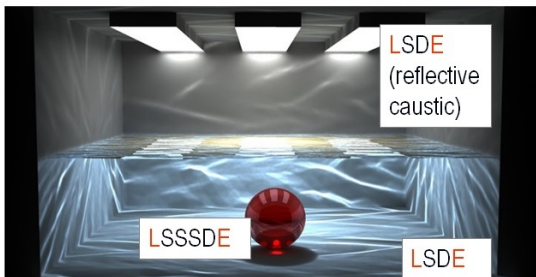




Notation for Light Transport



Example:





Example: Classification



Left : Recursive Ray Tracing (Whitted Ray Tracing) , **Right** : Global Solution

- ▶ Global Illumination: $L[D|S]*E$
- ▶ Recursive Ray Tracing: $L[DS*]E$

- ▶ Radiosity Method: $L[D]*E$
- ▶ Local Illumination: $L[D]E$



Implementation of Ray Tracers





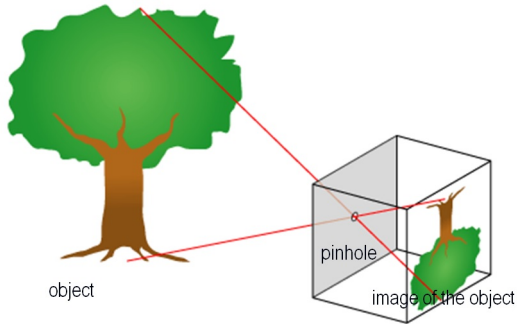
Software: How shall things be computed?

- ▶ Ray Tracing Core
 - ▶ Recursive/iterative computation of ray paths
 - ▶ Efficient ray-primitive intersection testing
- ▶ Material Shader
 - ▶ Implements material properties
- ▶ Light Shader
 - ▶ Implements lighting
- ▶ Image Reconstruction
 - ▶ Ray tracing is sample based: What is the optimal approach to reconstruct the final image? Anti-Aliasing?



Virtual Camera:

- ▶ Simulates behavior of real models
- ▶ Many virtual camera types thinkable
- ▶ In practice often simple pinhole cameras modeled



Pinhole camera



Ray Casting:¹⁰

- ▶ Simple approach to image synthesis
- ▶ Generates realistic raster (pixel) images
- ▶ Accounts for shadowing and occlusion

Algorithm Ray Casting

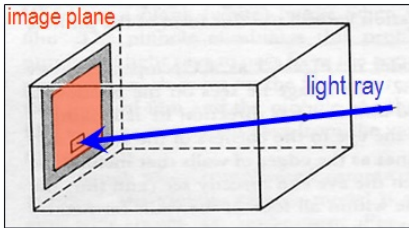
```
1: for each pixel  $p$  do
2:   Trace a ray from the eye “backwards” through the scene until it first hits geometry
3:   for each lightsource  $ls$  in the scene do
4:     Cast a shadow ray in direction to the light source  $ls$ 
5:     if Shadow ray does NOT hit any object then
6:       Some light gets reflected to eye according to material properties
7:     else
8:       No light gets reflected to eye
9:     end if
10:  end for
11: end for
```

¹⁰A. Appel. “Some Techniques for Shading Machine Renderings of Solids”. In: *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference*. ACM, 1968, pp. 37–45.

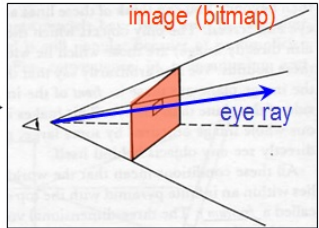


Ray Casting: Pinhole camera principle in classical ray casting

- ▶ Pinhole $\rightarrow$ eye
- ▶ Image plane = raster image = window to virtual world
- ▶ Light ray $\rightarrow$ eye ray



Pinhole camera

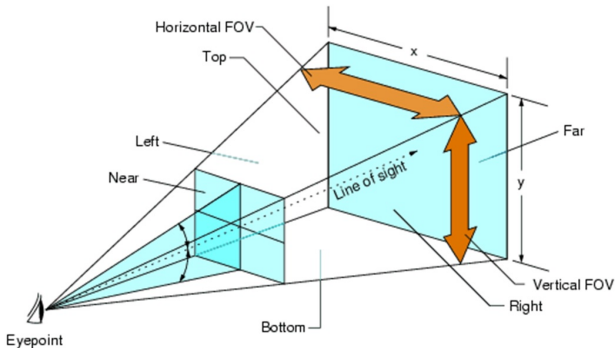


Typical ray caster



Ray Casting: Field of View

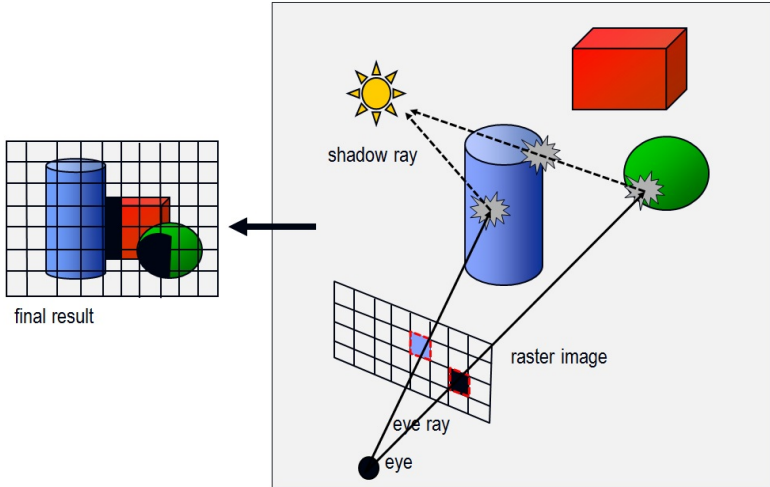
- ▶ Camera field of view defines extend of observable world
- ▶ Horizontal and vertical field of view (FOV) is typically given by two different angles
- ▶ FOV implicitly also determines image aspect ratio



$$\text{Aspect Ratio} = \frac{y}{x} = \frac{\tan(\text{vertical FOV}/2)}{\tan(\text{horizontal FOV}/2)}$$



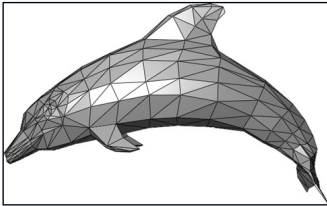
Ray Casting: Example



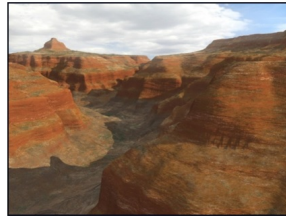


Question: How to represent geometry?

- ▶ Very often triangle meshes are used to specify the surface of a model
- ▶ Sometimes also sphere, cone, cylinder (or other quadrics) primitives
- ▶ Procedural geometry (e.g. fractal-based models) for landscapes, ...



Triangle mesh



Procedural geometry

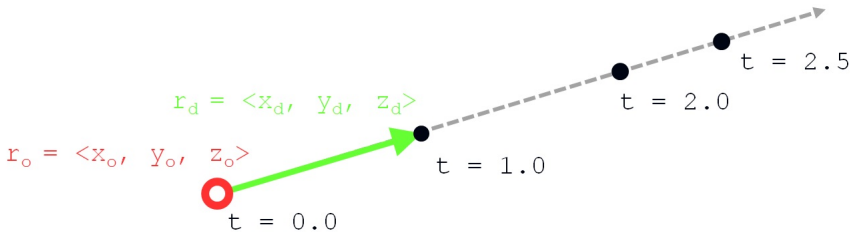


Question: How to generate/represent rays?

Ray expressed as function of a single parameter

$$\mathbf{r}(t) = \mathbf{r}_o + t \cdot \mathbf{r}_d$$

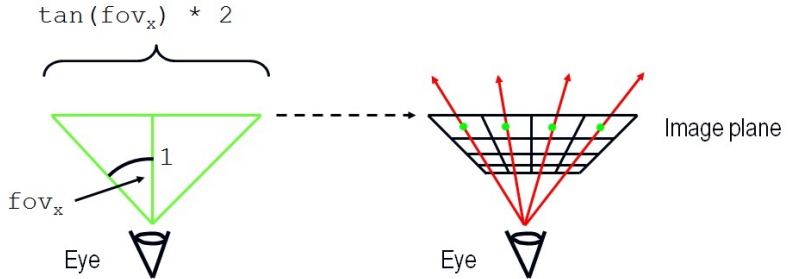
Note: Normally, it is assumed that $\|\mathbf{r}_d\| = 1$!





Question: How to generate/represent rays?

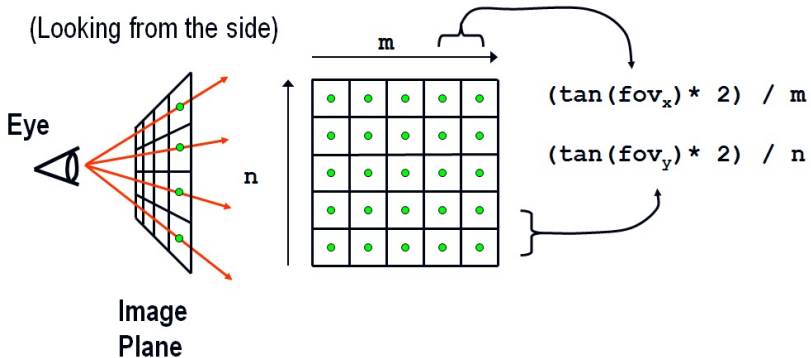
Trace a ray for each pixel in the image plane





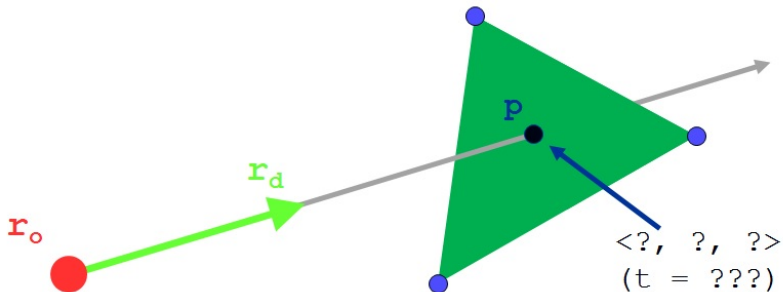
Question: How to generate/represent rays?

Trace a ray for each pixel in the image plane





Question: How to compute ray intersections with geometry?





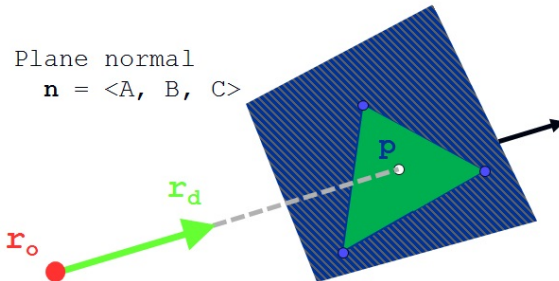
Question: How to compute ray intersections with geometry?

Step 1: Intersect ray with plane

$$\langle \mathbf{n} | \mathbf{x} \rangle + d = 0$$

Substituting $\mathbf{x}$ by parametric ray form $\mathbf{r}(t) = \mathbf{r}_o + t \cdot \mathbf{r}_d$ and solving for t yields to

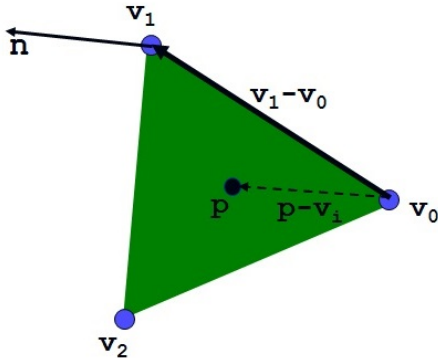
$$t = -\frac{\langle \mathbf{n} | \mathbf{r}_o \rangle + d}{\langle \mathbf{n} | \mathbf{r}_d \rangle}$$





Question: How to compute ray intersections with geometry?

Step 2: Is p inside the triangle? (check against all triangle edges)



Define indicator

$$E_i = \langle (v_{i+1} - v_i) \times (p - v_i) | n \rangle$$

Check sign of E_i for each edge

- Indicates if p is “left” or “right” to the i -th edge
- If signs are all positive or negative, point is inside triangle!



Question: How to compute ray intersections with geometry?

Now: We need to find the closest intersection point

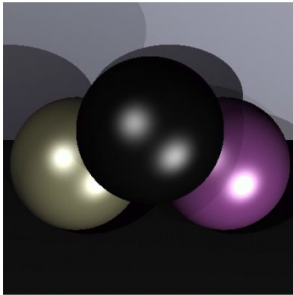
Algorithm Naive Intersection Finding

```
1: for each primitive in the scene do
2:   if ray intersects primitive then
3:     Compute intersection point  $p$ 
4:     if  $p < p_{min}$  then
5:       Store  $p_{min} = p$ 
6:       Store other important intersection information
7:     end if
8:   end if
9: end for
10: Return closest intersection  $p_{min}$  including all relevant information
```

Note: This is rather inefficient! Later, we will learn how to accelerate this.



Recursive Ray Tracing: Ray Casting simulates only direct illumination¹¹



local illumination method with shadow computation



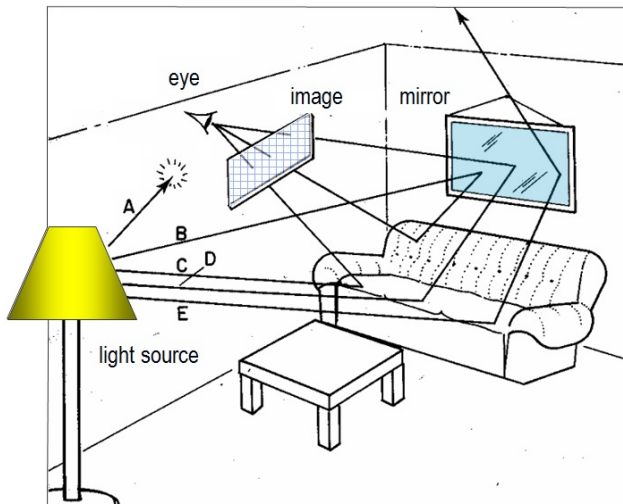
recursive ray-tracing

- ▶ Specular (“mirror-like”) inter-reflections are simulated
- ▶ Simulation is not complete though ...
- ▶ ... but forms the basis of most modern rendering systems

¹¹T. Whitted. “An Improved Illumination Model for Shaded Display”. In: *Commun. ACM* 23.6 (1980), pp. 343–349.



Recursive Ray Tracing: Example

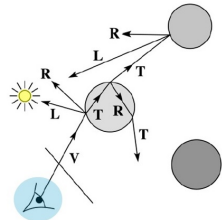




Algorithm Recursive Ray Tracing

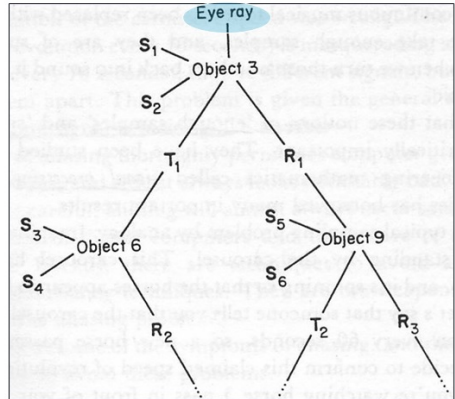
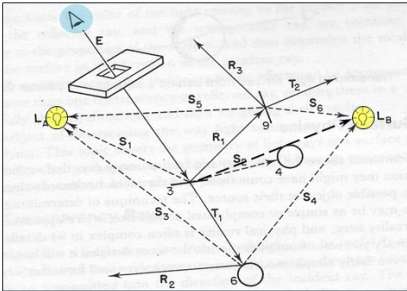
- 1: **for** each pixel p **do**
- 2: Trace a ray from the eye “backwards” through the scene until it first hits geometry
- 3: Shoot light rays l to all light source and compute reflected light
- 4: **if** material at intersection point is reflective **then**
- 5: Shoot a ray in ideal reflection direction r and recursively trace it
- 6: **end if**
- 7: **if** material at intersection point is refractive **then**
- 8: Shoot a ray in ideal refraction direction t and recursively trace it
- 9: **end if**
- 10: **end for**

$$L(x, v) = \underbrace{\sum_l f_{BRDF}(l, x, v) L_e(x_l, l)}_{\text{ray casting}} + \underbrace{k_{ir} L(x_r, r)}_{\text{ideal reflection}} + \underbrace{k_{it} L(x_t, t)}_{\text{ideal transmission}}$$





Recursive Ray Tracing: Ray Tree Example





Reflection and Refraction: Speed of light is a material dependent property (related to refractive index). For realistic materials it is also a spectral property!

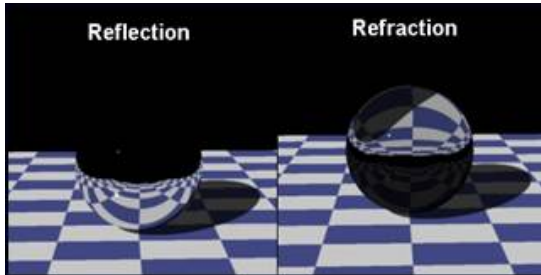




Reflection and Refraction: Speed of light is a material dependent property (related to refractive index). For realistic materials it is also a spectral property!

Reflection: Change of propagation direction of light (**without** transition to the other medium) at the interface of two media with varying refractive index

Refraction: Change of propagation direction of light (**with** transition to the other medium) at the interface of two media with varying refractive index





Reflection:

Law of reflection: $\theta_i = \theta_r$

Incident direction $\mathbf{d}$ and reflected direction $\mathbf{r}$ are coplanar

Consider the sum of the negative incident direction and reflection direction:

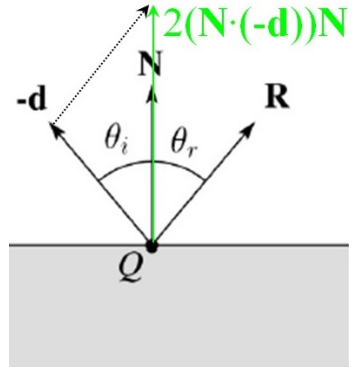
$$\mathbf{r} + (-\mathbf{d}) = 2 \langle \mathbf{n} | -\mathbf{d} \rangle \mathbf{n}$$

Solving for $\mathbf{r}$ leads to

$$\mathbf{r} = \mathbf{d} - 2 \langle \mathbf{n} | \mathbf{d} \rangle \mathbf{n}$$

Note: An equivalent (but computationally slower!) representation is given by the Householder Matrix:

$$\mathbf{r} = (\mathbf{I} - 2 \mathbf{n} \mathbf{n}^T) \mathbf{d}$$





Refraction:

Law of refraction:

$$\frac{\sin(\theta_i)}{\sin(\theta_t)} = \frac{n_2}{n_1} =: \eta$$

Incident direction $\mathbf{d}$ and refracted direction $\mathbf{t}$ are coplanar with refractive indices n_1 and n_2

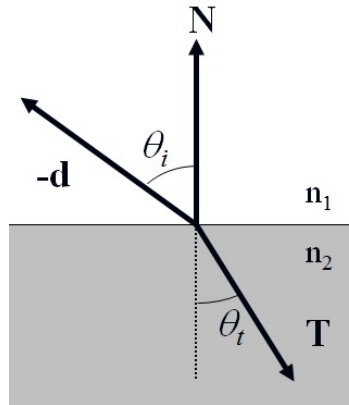
The refraction direction is given by

$$\mathbf{t} = c_n \mathbf{n} + c_d \mathbf{d}$$

with coefficients

$$c_n = -\frac{1}{\eta} \langle \mathbf{n} | \mathbf{d} \rangle - \sqrt{1 - \frac{1}{\eta^2} (1 - \langle \mathbf{n} | \mathbf{d} \rangle^2)}$$

$$c_d = \frac{1}{\eta}$$





Refraction:

The refraction direction is given by

$$\mathbf{t} = c_n \mathbf{n} + c_d \mathbf{d}$$

with coefficients

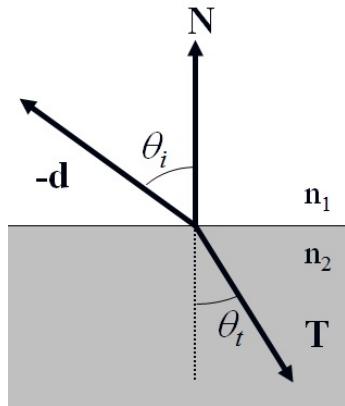
$$c_n = -\frac{1}{\eta} \langle \mathbf{n} | \mathbf{d} \rangle - \sqrt{1 - \frac{1}{\eta^2} (1 - \langle \mathbf{n} | \mathbf{d} \rangle^2)}$$

$$c_d = \frac{1}{\eta}$$

Note: The refraction direction only exists if

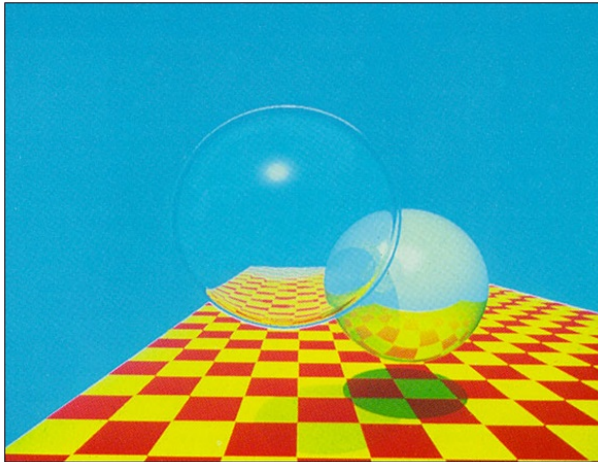
$$1 - \frac{1}{\eta^2} (1 - \langle \mathbf{n} | \mathbf{d} \rangle^2) \geq 0$$

Otherwise, there is only a reflection (Total internal reflection)!





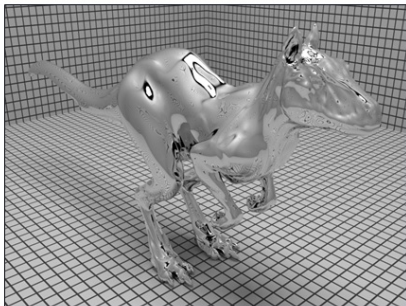
Recursive Ray Tracing: Results



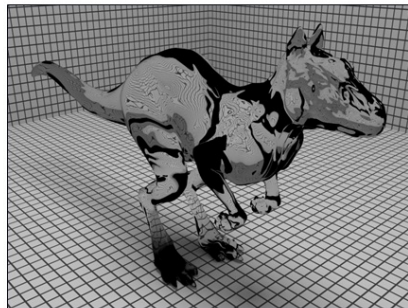
Turner Whitted, 1980



Recursive Ray Tracing: Results



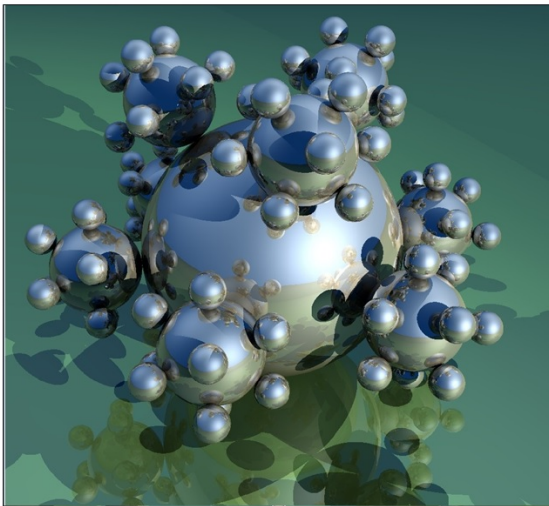
specular reflection



specular refraction

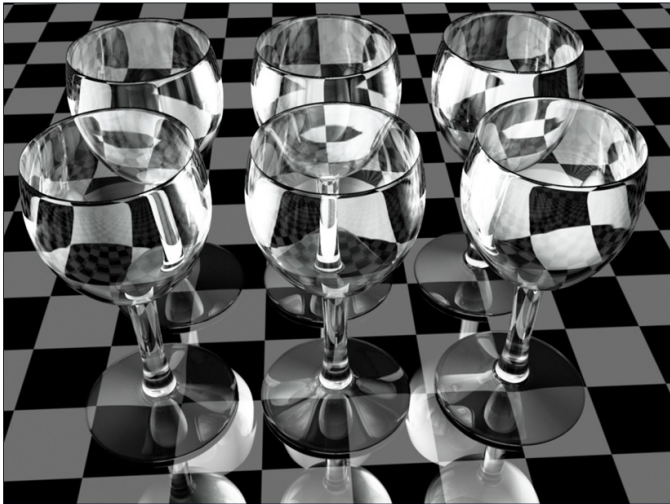


Recursive Ray Tracing: Results





Recursive Ray Tracing: Results





- [1] A. Appel. "Some Techniques for Shading Machine Renderings of Solids". In: *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference*. ACM, 1968, pp. 37–45.
- [2] M. F. Cohen, S. E. Chen, J. R. Wallace, and D. P. Greenberg. "A progressive refinement approach to fast radiosity image generation". In: *Proceedings of the 15th annual conference on Computer graphics and interactive techniques*. 1988, pp. 75–84.
- [3] C. M. Goral, K. E. Torrance, D. P. Greenberg, and B. Battaile. "Modeling the interaction of light between diffuse surfaces". In: *ACM SIGGRAPH computer graphics* 18.3 (1984), pp. 213–222.
- [4] P. S. Heckbert. "Adaptive Radiosity Textures for Bidirectional Ray Tracing". In: *Proceedings of the 17th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '90. ACM, 1990, pp. 145–154.
- [5] J. T. Kajiya. "The Rendering Equation". In: *Proceedings of the 13th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '86. ACM, 1986, pp. 143–150.



- [6] R. Kuchar, T. Schairer, and W. Straßer. “Photorealistic Real-Time Visualization of Cultural Heritage: A Case Study of Friedrichsburg Castle in Germany.”. In: *Eurographics (Cultural Heritage)*. 2007, pp. 9–16.
- [7] F. E. Nicodemus, J. C. Richmond, J. J. Hsia, I. W. Ginsberg, and T. Limperis. *Geometrical considerations and nomenclature for reflectance*. Vol. 160. US Department of Commerce, National Bureau of Standards Washington, USA, 1977.
- [8] M. Pharr, W. Jakob, and G. Humphreys. *Physically based rendering: From theory to implementation*. Morgan Kaufmann, 2016. URL: <http://www.pbr-book.org/>.
- [9] L. Szirmay-Kalos. *Monte Carlo Methods in Global Illumination - Photo-realistic Rendering with Randomization*. Saarbrücken, Germany, Germany: VDM Verlag, 2008.
- [10] L. Szirmay-Kalos. *Monte Carlo Methods in Global Illumination - Script WS 1999/2000*. Vienna University of Technology, 2000.
- [11] T. Whitted. “An Improved Illumination Model for Shaded Display”. In: *Commun. ACM* 23.6 (1980), pp. 343–349.